



A Consortium for CONvection-scale modelling
Research and Development

Stay report

Towards a more portable EPyGrAM (FA, ectrans, PyPI)

Realized by : CHIKHI Walid

National Office of Meteorology

ALGERIA

Supervised by : Alexandre MARY

CNRM / Meteo France

FRANCE

CNRM/GMAP - Toulouse

May 27 - 08 June , 2024

Contents:

Introduction

1. EPyGrAM

2. Python Package EPyGrAM on PYPI

3. Externalization of FA-LFI-LFA formats from IAL code

3.1. FA-LFI-LFA package in python (falfilfa4py) :

3.2. EPyGrAM and falfilfa4py :

4. Spectral transforms

5. Cleaning FA-LFI-LFA code :

6. Summary of paths and GitHub repositories

Conclusion & perspectives

Introduction

During my recent stay at Météo-France from May 27 to June 8, 2024, I focused on enhancing the EPyGrAM code, a Python package used for visualizing atmospheric model data. My work involved adapting EPyGrAM to better meet research needs and completely removing its dependency on IAL code. This included making the package easier to install with PIP and ensuring compatibility with various atmospheric data formats across multiple operating systems.

This report details the modifications and improvements made to EPyGrAM, including:

- Complete reconstruction and make it easily installable with pip.
- Management of FA-LFI-LFA formats and spectral transforms within EPyGrAM through new Python packages.
- Cleanup of FA-LFI-LFA code.

I hope this report reflects the collaborative efforts and innovations that contribute to advancing atmospheric sciences through EPyGrAM within the ACCORD consortium.

1. **EPyGrAM “Enhanced Python for Graphics and Analysis of Meteorological fields” :**

EPyGrAM is a Python package designed for handling meteorological fields and various data formats from which these fields can be extracted, such as FA, LFI, LFA, GRIB, and NetCDF. It provides user-friendly interfaces for processing data from Numerical Weather Prediction (NWP), focusing on simple access to data and metadata. EPyGrAM is built around three core concepts: fields (data with metadata), geometries (spatial descriptions), and resources (aggregations of fields in specific formats). The package includes a set of tools and classes for easy data manipulation and is designed for use by anyone building meteorological applications in Python.

Overall, EPyGrAM is designed to make reading and writing fields from/to resources straightforward, with additional features for working with fields and geometries detailed in its documentation : [EPyGrAM Docs](#) .

2. Python Package EPyGrAM on PYPi :

The first step of the stay involves making the current version of the software as a python package fully installable (except FA/LFI/LFA & spectral transforms support) using pip and releasing an official version on PyPI. To achieve this, a significant restructuring of the EPyGrAM source code was necessary. During the initial development of EPyGrAM, the codebase was divided into two almost distinct and separate packages: the core of EPyGrAM and additional functionalities like the management of internal Météo-France formats (FA, LFI, LFA) and spectral transforms.

To address this, a "src layout" organization was adopted during the architectural modifications, merging both parts into a single structure. The core functionalities were placed in a unified directory `src/epygram`, while the additional features were organized into a new `epygram/extra` folder. This restructuring not only streamlined the codebase but also made it more coherent and easier to manage, setting the stage for broader distribution and use through pip and PyPI.

The changes made are summarized in this patch : [EPyGrAM restruction changes](#)

After restructuring EPyGrAM, the next phase of the project focused on preparing the package for distribution on PyPI (Python Package Index). This involved incorporating necessary configuration ([pyproject.toml](#)) and installation files ([setup.py](#)) to streamline deployment. Configuration files were set up to define package metadata, dependencies, and entry points, while installation scripts were developed to automate setup tasks on user systems.

setup.py :

```
from setuptools import setup, find_packages
setup(
    scripts = [
        'bin/ddhlfa_plot.py',
        'bin/domain_maker.py',
        'bin/epy_cartoplot.py',
        'bin/epy_conv.py',
        'bin/epy_delfield.py',
        'bin/epy_doc.py',
        'bin/epy_hist.py',
        'bin/epy_movefield.py',
        'bin/epy_plot3d.py',
        'bin/epy_point.py',
```

```

        'bin/epy_profile.py',
        'bin/epy_section.py',
        'bin/epy_spectrum.py',
        'bin/epy_stats.py',
        'bin/epy_what.py',
        'bin/fa_sp2gp.py',
        'bin/run_epyweb.py',
        'bin/what_the_grib.py',
    ]
)

```

pyproject.toml :

```

[build-system]
requires = ["setuptools", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "epygram"
dynamic = ["version"]
description = "EPYGrAM : Enhanced Python for Graphics and Analysis of Meteorological fields"
authors = [
    { name = "Alexandre MARY", email = "alexandre.mary@meteo.fr" },
    { name = "Sébastien Riette" , email = "sebastien.riette@meteo.fr"}
]
maintainers = [
    { name = "Alexandre MARY", email = "alexandre.mary@meteo.fr" },
    { name = "Sébastien Riette" , email = "sebastien.riette@meteo.fr"}
]

readme = "README.md"
license = {file = "LICENSE.txt"}
keywords = ["NWP", "meteorology", "GRIB", "FA"]

classifiers = [
    'Development Status :: 5 - Production/Stable',
    'Intended Audience :: Science/Research',
    'Intended Audience :: Developers',
    'Programming Language :: Python',
    'Programming Language :: Python :: 3.6',
    'Programming Language :: Python :: 3.7',
    'Programming Language :: Python :: 3.8',
    'Programming Language :: Python :: 3.10',
    'Programming Language :: Python :: 3.11',

```

```

    'Programming Language :: Python :: 3.12',
    'Operating System :: Unix',
]
requires-python = ">=3.6"
dependencies=[
    "footprints",
    "bronx",
    "numpy",
    "cartopy",
    "matplotlib",
    "eccodes",
    "netCDF4",
    "pyproj",
    "six",
    "pyyaml", # issue with bronx !!
    "ctypesForFortran", # temporary, until complete externalisation of arpifs4py
]
[tool.setuptools]
include-package-data = true

[tool.setuptools.packages.find]
where = ["src"]

[tool.setuptools.dynamic]
version = {attr = "epygram.__version__"}
[project.optional-dependencies]
docs = [
    "sphinx",
]
graphics = [
    "vtk",
    "contourpy",
]
features=[
    "taylorism",
    "pyresample",
    "scipy",
    "web.py"
]
format=[
    "h5py",
    "pillow"
]

vortex = [
    "vortex"
]

```

```

all = [
    "taylorism",
    "footprints",
    "bronx",
    "numpy",
    "cartopy",
    "matplotlib",
    "eccodes",
    "vtk",
    "vortex",
    "pillow",
    "contourpy",
    "h5py",
    "netCDF4",
    "pyproj",
    "pyresample",
    "scipy",
    "six",
    "web.py"
]

[project.urls]
documentation = "https://umr-cnrm.github.io/EPyGrAM-doc"
source = "https://github.com/UMR-CNRM/EPyGrAM.git"
download = "https://github.com/UMR-CNRM/EPyGrAM/releases"
tracker = "https://github.com/UMR-CNRM/EPyGrAM/issues"

[tool.pytest.ini_options]
pythonpath = [
    "src",
]
testpaths = [
    "tests"
]

```

The Python package EPyGrAM (version 1.50 and 1.5.1) has been uploaded to the PyPI repository under the **CNRM account**. You can find the package at the following link: [EPyGrAM PyPI](https://pypi.org/project/EPyGrAM/)

EPyGrAM could possibly be installed with multiple options and features :

options	Installed package
default	footprints bronx numpy cartopy matplotlib eccodes netCDF4 pyproj six pyyaml # issue with bronx !!
docs	default + sphinx
graphics	default + : vtk contourpy
features	default + : taylorism pyresample scipy web.py"
format	default + : h5py pillow
vortex	default + vortex
all	taylorism footprints bronx numpy cartopy matplotlib eccodes vtk vortex pillow contourpy h5py

	netCDF4 pyproj pyresample scipy six web.py
--	---

To install EPyGrAM on a Linux architecture platform, you can use the following command:

```
pip3 install epygram
```

Or install all features :

```
pip3 install epygram[all]
```

3. Externalisation of formats FA - LFI - LFA from IAL :

This part is the subject of my first scientific stay at Météo-France, and the report can be checked from the following link : [LFIFA_chikhiw_stay_report_2023](#) .

3.1 FA-LFI-LFA package in python (**falfilfa4py**) :

Although the LFIFA package has been successfully pre-externalized (the code still lies in IAL repo, but has been synchronized in an external repo and will be removed from IAL in CY50T1), the component serving as wrappers between the Fortran and C routines, encapsulated in the shared library (libfa_dp.so), still resides within the EPyGrAM codebase. This component not only needs to be compiled but also requires all dependencies such as eccodes, fiat, and gribex to be present during installation. Because of the challenging and labor-intensive nature of this task, Alexandre proposed the creation of a dedicated Python package solely for handling fa-lfi-lfa formats, including all necessary dependencies. This package will be shared on the PyPI repository.

One of the challenges encountered involved the ``ctypesforfortran.F90`` module, which was used to facilitate the interfacing of FORTRAN code with Python by compiling and assembling the shared library ``libs4py.so``. Now that we are dividing the library into its two main functions for handling FA-LFI-LFA formats and spectral transforms, both libraries will require the ``ctypesforfortran`` module.

Following this, Alex took on the responsibility of externalizing ``ctypesforfortran.py`` into a simple Python package installable via pip. Additionally, the ``epygram/extra/arpifs4py`` directory was split into two separate directories: ``falfilfa4py`` and ``ialsptans4py``, both ready to be fully externalized.

The next step involved integrating Python wrappers into the LFIFA code, making it compatible with

Python 3, and generating a wheel that links the `libfa\_dp.so` library to the wrappers. To accomplish this, preparation work for the package began by extracting code specific to these formats from IAL, combining it with Python wrappers, and compiling it within a **Docker manylinux2014** container. This container was based on **RedHat 5**, using an older **glibc** version (**2.17**) to ensure reverse compatibility. The build process utilized CMake and the Python package scikit-build. This setup supports multiple Python versions ranging from 3.6 to 3.12. The modifications (red files and directories) made can be summarized as follows :

LFIFA code tree :

```
|— cmake
... ( Nothing changed )
|— CMakeLists.txt
|— falfilfa4py ( Code From EPyGrAM )
|   |— FA.py
|   |— __init__.py
|   |— LFA.py
|   |— LFI.py
|— MANIFEST.in
|— pyproject.toml
|— README.md
|— setup.py
|— src      ( Nothing changed )
...
|— VERSION
```

CMakeLists.txt :

```
....
if(SKBUILD)
file(GLOB PYTHON_SETUP_FILES setup.py)
endif()
....
```

pyproject.toml :

```
[build-system]
requires = ["setuptools", "wheel", "scikit-build"]
build-backend = "setuptools.build_meta"
```

setup.py :

```
from skbuild import setup

setup(
    name="falfilfa4py",
    version="0.0.1",
    packages=['falfilfa4py'],
    cmake_install_dir="falfilfa4py",
    cmake_minimum_required_version="3.13",
    setup_requires=["scikit-build", "setuptools"],
    install_requires=["numpy" , "ctypesForFortran"],# Add other Python dependencies here
)
```

MANIFEST.in :

```
include CMakeLists.txt
include README.md
include VERSION
include pyfa/*.py
include pyproject.toml
include setup.py
recursive-include src *
```

3.2 EPyGrAM and falfilfa4py :

The next step during my stay was to integrate EPyGrAM with the newly developed package `falfilfa4py`. This process involved not only adding `falfilfa4py` as a dependency to the EPyGrAM package but also ensuring that `falfilfa4py` is established as an external, independent package. Currently, both `falfilfa4py` and the modified EPyGrAM are available exclusively on test.pypi. This allows for thorough testing, overseen by Alexandre, to ensure their functionality and stability.

4. Spectral transforms :

Due to time constraints, the task of LAM spectral transforms management within EPyGrAM was

addressed after my stay. The part of the work concerning spectral transforms was managed by Daan Degrauwe from RMI, who completed the integration of LAM spectral transforms into the ectrans version from IAL code. With this task completed, the next step for EPyGrAM is to externalize this last dependency on IAL.

Similar to the approach taken with `falfilfa4py`, initial testing was conducted by incorporating the wrappers and combining everything with the library, `libtrans\_dp.so`. A cleaner and more refined version will be prepared soon.

5. Cleaning FA-LFI-LFA code :

During my time at Météo-France, I, along with Alexandre Mary and Ryad El Khatib, initiated discussions about cleaning and modifying the LFIFA source code before its externalization. The modifications discussed include:

- The `grib\_api\_interface.F90` module (Ref : report stay 2023)
- Redundancies in declarations between `lfi\_precision` and `parkind1`
- Removal of `gribex`, which is no longer used in IAL
- Externalization of Namelist for handling GRIB options
- Abort management .

Few tasks from the list were initiated during my stay but remain unfinished. We are committed to completing these tasks and contributing to the upcoming IAL cycles.

6 . Summary of paths and github repositories :

<i>Package</i>	<i>State</i>	<i>version</i>	<i>path and repositories</i>	<i>PyPI</i>
EPyGrAM	stable	1.5.0	EPyGrAM_github_stable	PyPi
EPyGrAM	Dev	1.5.1	EPyGrAM_github_falfilfa4py_extern	Test PyPi
LFIFA	Dev	/	LFIFA_github	Test PyPi
falfilfa4py	Dev	0.1.4	falfilfa4py_github_dev	Test PyPi
ectrans + lam	Dev	/	ialsprtrans4py_github	NULL

Conclusions and perspectives :

In conclusion, my stay at Météo-France has been crucial in advancing the integration and enhancement of EPyGrAM, particularly by making it easier to install and moving it further toward independence from IAL. This was achieved through the development of three additional Python

packages and dependencies for EPyGrAM (1 official release : ctypesforfortran and 2 still under development : falfilfa4py and ialsptrans4py) . We are dedicated to contributing significantly to future versions of EPyGrAM. Looking ahead, our focus will remain on refining and optimizing the LFIFA code, ensuring robustness and compatibility across different environments. We aim to make substantial contributions to CY50 or CY50T1, depending on our progress.

Although we were unable to complete all tasks by the end of the stay due to encountered obstacles, we have continued the work remotely to ensure ongoing progress. These efforts highlight our commitment to advancing EPyGrAM as a valuable tool for researchers and meteorologists, fostering collaboration and innovation in atmospheric sciences.

Acknowledgments:

I want to express my gratitude to Alexandre MARY for his exceptional support and guidance during my stay. I also want to thank all the GMAP staff for the warm reception, especially : Thibault Lestang, Claude Fischer, Eric Éscalrière, and Cecile loo .